

Arhitectura calculatoarelor

Curs 5

5. Extinderea procesorului. Aritmetica calculatoarelor, unități aritmetice și logice

Cristian Vancea

<https://users.utcluj.ro/~vcristian/AC.html>

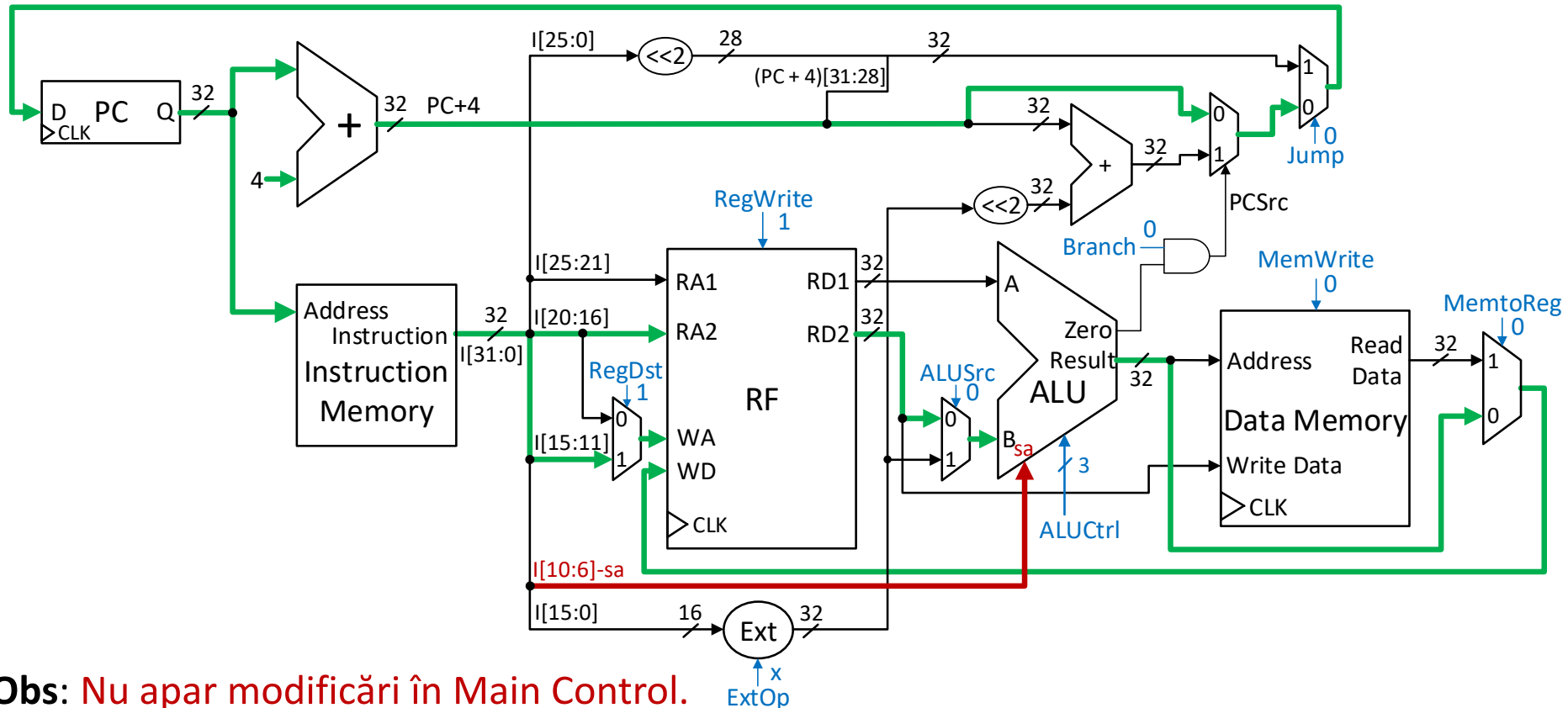
Objective

1. Extinderea procesorului
2. Reprezentarea numerelor în MIPS
3. Aritmetica în Complement față de 2
4. Aritmetica în MIPS
5. Proiectarea ALU

5.1 Extinderea procesorului

- SLL**: $RF[rd] \leftarrow RF[rt] \ll sa$ **SRL**: $RF[rd] \leftarrow RF[rt] \gg sa$ **SRA**: $RF[rd] \leftarrow RF[rt] \gg_a sa$
 toate 3 de tip R

000000	xxxxx	rt(5b)	rd(5b)	sa(5b)	func(6b)
--------	-------	--------	--------	--------	----------



Obs: Nu apar modificări în Main Control.

Opcode	RegDst	RegWrite	ALUSrc	ExtOp	ALUOp _{1:0}	MemWrite	MemtoReg	Branch	Jump
000000 (tip R)	1	1	0	x	10 (R)	0	0	0	0

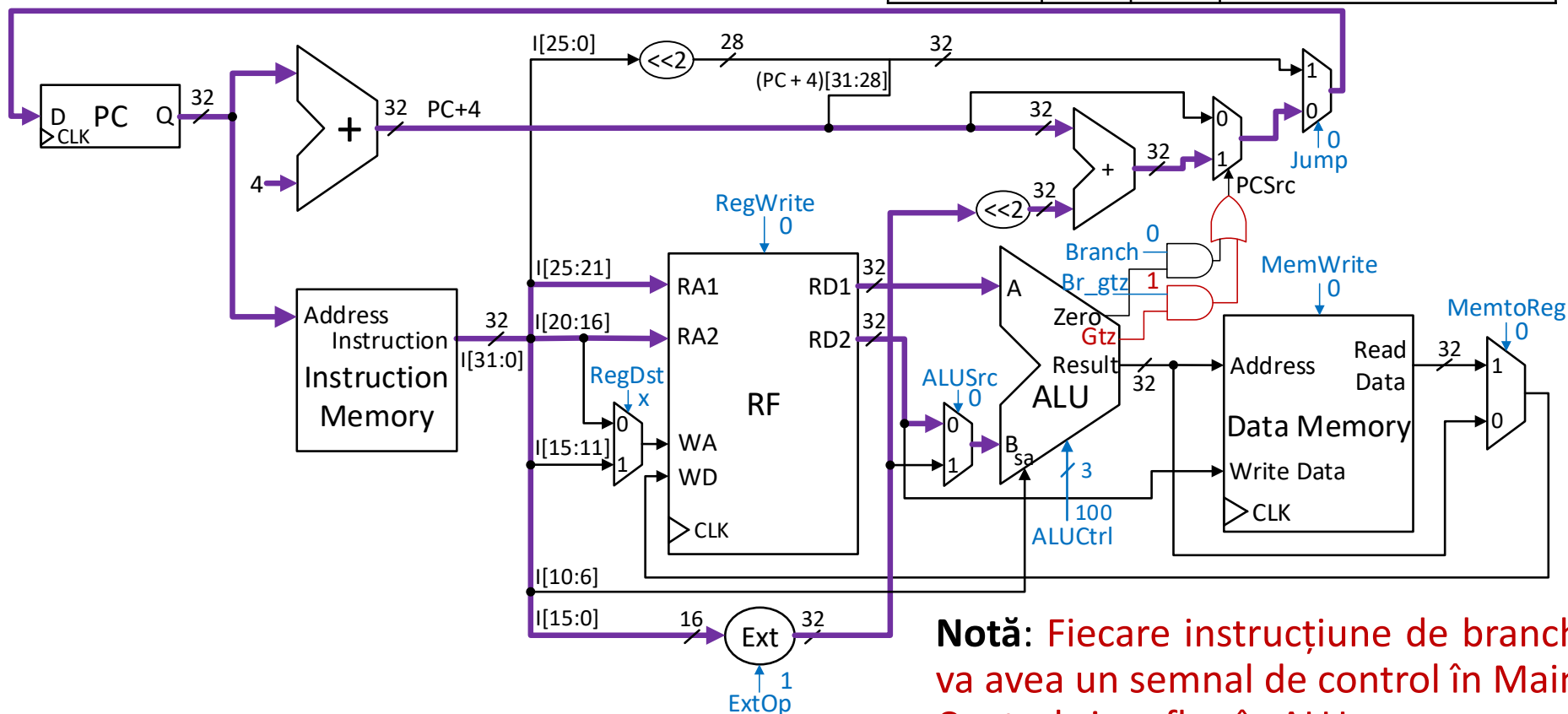
Important: Se alocă noi coduri ALU Ctrl pentru fiecare operație de deplasare.

5.1 Extinderea procesorului

- BGTZ (BGTZ \$rs)**: if RF[rs]>0 then $PC \leftarrow (PC+4) + S_Ext(offset) \ll 2 \rightarrow$ de tip I

else $PC \leftarrow PC+4$

000111	rs(5b)	00000	offset (16b)
--------	--------	-------	--------------



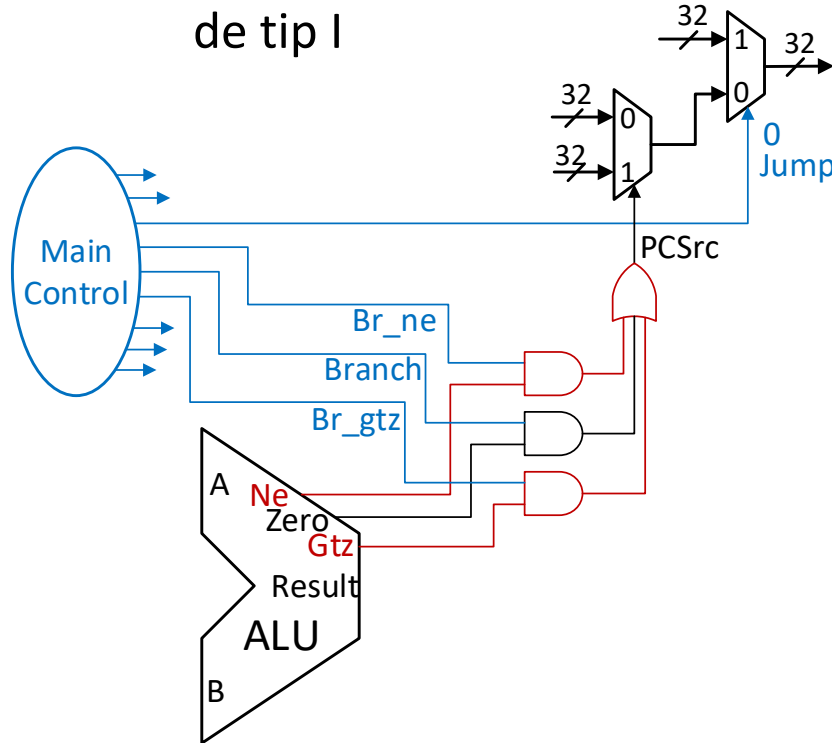
Notă: Fiecare instrucțiune de branch va avea un semnal de control în Main Control și un flag în ALU.

Instr.	RegDst	RegWrite	ALUSrc	ExtOp	ALUOp _{1:0}	MemWrite	MemtoReg	Branch	Br_gtz	Jump
BGTZ	x	0	0	1	01 (-)	0	x	0	1	0
BEQ	x	0	0	1	01 (-)	0	x	1	0 ₄	0

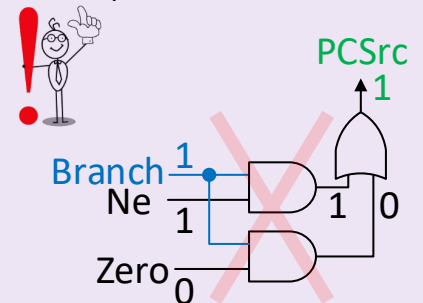
Important: Br_gtz se activează numai pt. BGTZ, Branch se activează numai pt. BEQ.

5.1 Extinderea procesorului

Mai multe instrucțiuni de branch: **BEQ**, **BGTZ**, **BNE**.
de tip I



De ce nu funcționează folosirea unui semnal comun **Branch** la mai multe instrucțiuni de salt:



Presupunem: \$t1=6, \$t2=7
La execuția: BEQ \$t1, \$t2, 3
se face un salt nedorit,
deoarece **PCSrc=1**.

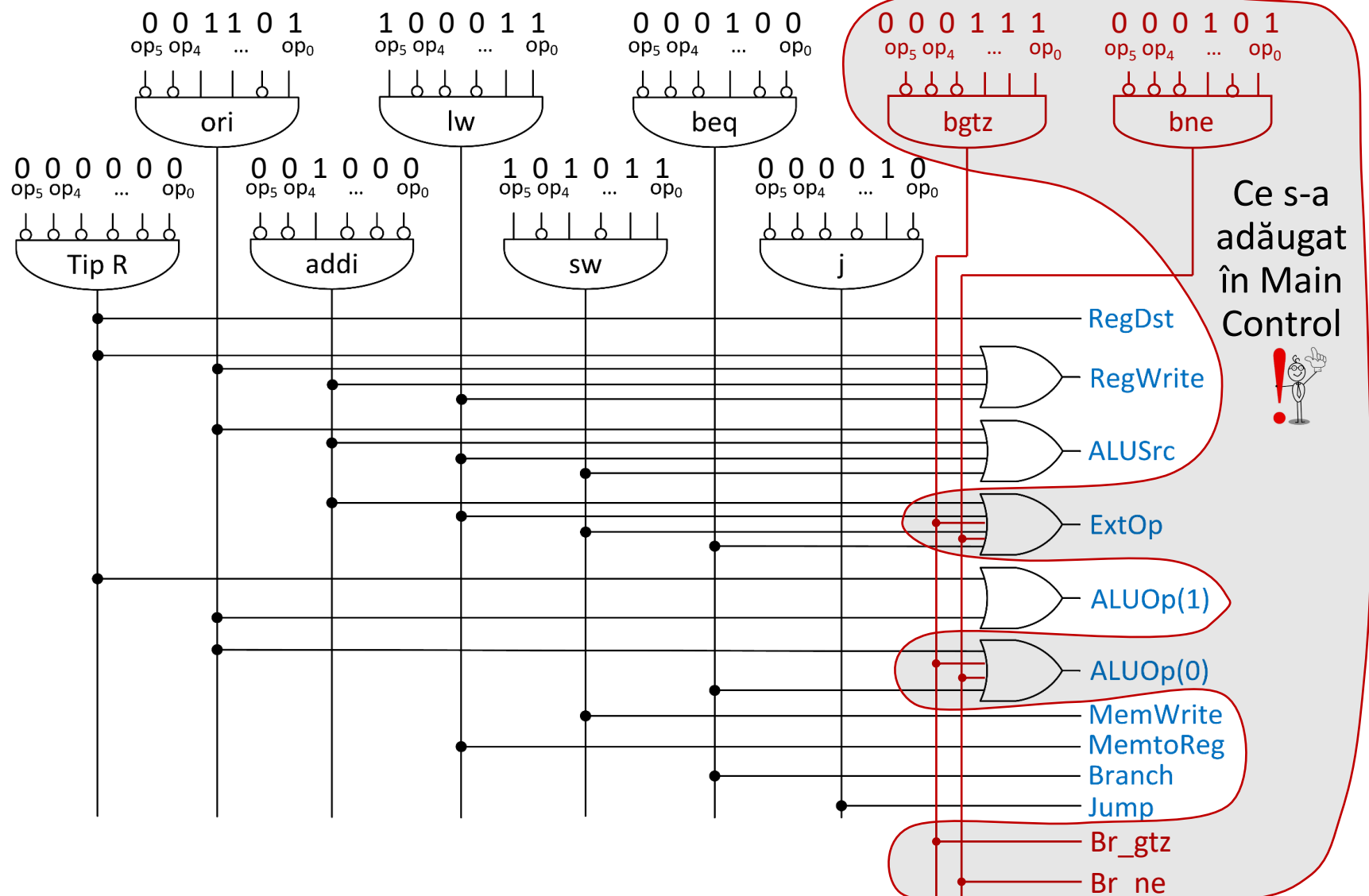


Fiecare instrucțiune de branch are asociat un **semnal de control** în **Main Control** (activat numai în prezența instrucțiunii) și un flag de validare în ALU.

Instr.	RegDst	RegWrite	ALUSrc	ExtOp	ALUOp _{1:0}	MemWrite	MemtoReg	Branch	Br_gtz	Br_ne	Jump
BGTZ	x	0	0	1	01 (-)	0	x	0	1	0	0
BEQ	x	0	0	1	01 (-)	0	x	1	0	0	0
BNE	x	0	0	1	01 (-)	0	x	0	0	1	0

- **Main Control:** inserare semnale de control pentru **BGTZ**, **BNE**.

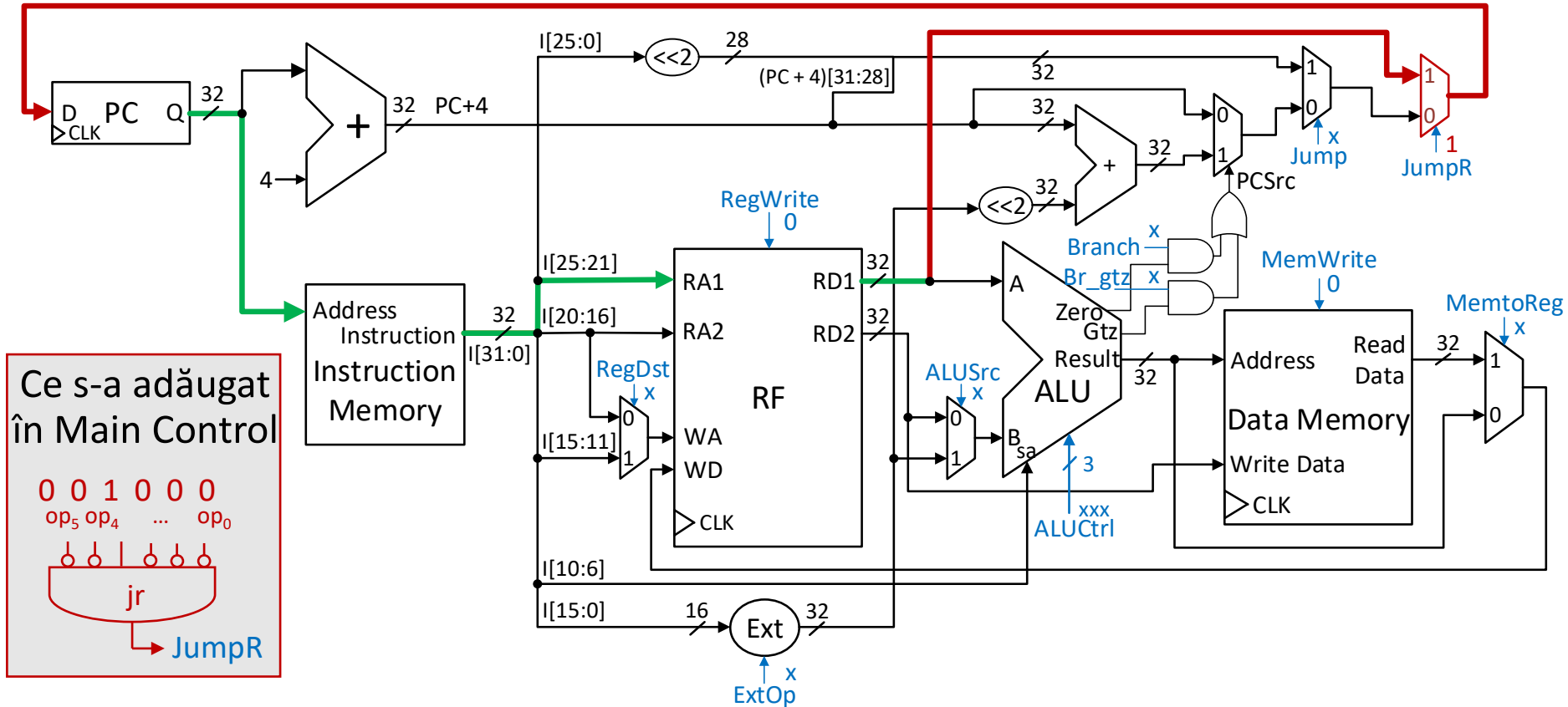
Opcode	RegDst	RegWrite	ALUSrc	ExtOp	ALUOp _{1:0}	MemWrite	MemtoReg	Branch	Br_gtz	Br_ne	Jump
000111(BGTZ)	x	0	0	1	01 (-)	0	x	0	1	0	0
000101(BNE)	x	0	0	1	01 (-)	0	x	0	0	1	0



5.1 Extinderea procesorului

001000	rs(5b)	xxxxx	xxxxxxxxxxxxxxxxxxxxx
--------	--------	-------	-----------------------

- JR** (Jump Register: JR \$rs): $PC \leftarrow RF[rs]$, implementare de tip I, cod 001000



Notă: Se adaugă la Main Control un semnal de control (**JumpR**) asociat pentru JR.

Instr.	RegDst	RegWrite	ALUSrc	ExtOp	ALUOp _{1:0}	MemWrite	MemtoReg	Branch	Br_gtz	Jump	JumpR
JR	x	0	x	x	xx	0	x	x	x	x	1
J	x	0	x	x	xx	0	x	x	x	1	0

Important: **JumpR** se activează numai pentru JR.

000011	address(26b)
--------	--------------

Notă: Se adaugă la Main Control un semnal de control (JumpL) asociat pentru JAL. Pentru JAL se activează Jump (dirijează saltul), JumpL și RegWrite (dirijează salvarea PC+4).

Instr.	RegDst	RegWrite	ALUSrc	ExtOp	ALUOp _{1:0}	MemWrite	MemtoReg	Branch	Br_gtz	Jump	JumpR	JumpL
JAL	x	1	x	x	xx	0	x	x	x	1	0	1
JR	x	0	x	x	xx	0	x	x	x	x	1 ₈	0
J	x	0	x	x	xx	0	x	x	x	1	0	0

5.2 Reprezentarea numerelor în MIPS

Întregi fără semn

- Se folosește baza 2.
- Conversia în baza 10: $N_{10} = \sum_{i=0}^{n-1} b_i \cdot 2^i$.
- Domeniu de reprezentare: $[0, 2^{n-1}]$.
- Apare la operații fără semn.

Întregi cu semn

- Se folosește reprezentarea binară în complement față de 2 (C_2).
- Conversia în baza 10: $N_{10} = -b_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i$.
- Domeniu de reprezentare: $[-2^{n-1}, 2^{n-1}-1]$.
- Apare la operații cu semn.
- Interpretarea pe 32 de biți:

pozitiv {

- 0000 0000 0000 0000 0000 0000 0000 0000 = 0_{10}
- 0000 0000 0000 0000 0000 0000 0000 0001 = 1_{10}
- 0000 0000 0000 0000 0000 0000 0000 0010 = 2_{10}
- ...
- 0111 1111 1111 1111 1111 1111 1111 1111 = $2,147,483,647_{10} = 2^{31}-1$ – maxim

negativ {

- 1000 0000 0000 0000 0000 0000 0000 0000 = $-2,147,483,648_{10} = -2^{31}$ – minim
- 1000 0000 0000 0000 0000 0000 0000 0001 = $-2,147,483,647_{10}$
- ...
- 1111 1111 1111 1111 1111 1111 1111 1110 = -2_{10}
- 1111 1111 1111 1111 1111 1111 1111 1111 = -1_{10}

↑ bitul de semn

5.3 Aritmetica în Complement față de 2

Formula de calcul C_2 : $\bar{\bar{N}}_2 = 2^n - N_2 = \bar{N}_2 + 1$, $\bar{N}_2 \Leftrightarrow$ inversarea biților

Ex. pe 8 biți: $7 = 00000111_2 \Rightarrow \bar{\bar{7}} = \overline{00000111}_2 + 1 = 11111000_2 + 1 = 11111001_2 = -2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^0 = -128 + 64 + 32 + 16 + 8 + 1 = -7$

Adunarea

Obs: $\bar{\bar{N}} = -N$

- Se adună biții de același rang cu transportul generat la rangul anterior. Biții de semn intră și ei în calcul.

Ex: Transportul $\Rightarrow$ 000011110

13 $\Rightarrow$ 00001101+ A
7 $\Rightarrow$ 00000111 B
20 $\leq$ 00010100 S

111110010

13 $\Rightarrow$ 00001101+ A
-7 $\Rightarrow$ 11111001 B
6 $\leq$ 00000110 S

Scăderea

- Se poate realiza ca o adunare cu complementul față de 2 al scăzătorului.

$$A - B = A + (-B) = A + \bar{\bar{B}}$$

$$\text{Ex: } 13 - 7 = 13 + \bar{\bar{7}} = 6$$

- Obs:** În hardware, la scădere se folosește tot un sumator.

5.3 Aritmetica în Complement față de 2

Depășirea la adunare cu semn

- Apare atunci când rezultatele depășesc domeniul de reprezentare.
 - Strategie de detecție: **Când valoarea rezultată are semnul incorect.**
 - Cazuri posibile:
 - 2 numere pozitive => suma este negativă
 - 2 numere negative => suma este pozitivă

Obs: Depășirea nu apare când numerele au semne diferite.

Ex. pe 4 biți:

Transportul ➡ 01110

3 => 0011+ A

7 => 0111 B

-6 <= 1010 S

10010

-3 => 1101+ A

-7 => 1001 B

6 <= 0110 S

- Generează un rezultat eronat care trebuie semnalizat de procesor.

	A	B	Condiție depășire	A_{n-1}	B_{n-1}	S_{n-1}	← biții de semn
A + B	≥ 0	≥ 0	$S < 0$	0	0	1	
	< 0	< 0	$S \geq 0$	1	1	0	
	≥ 0	< 0	nu e cazul				
	< 0	≥ 0	nu e cazul				

↓ Ovr = Overflow

$$Ovr = \bar{A}_{n-1} \cdot \bar{B}_{n-1} \cdot S_{n-1} + A_{n-1} \cdot B_{n-1} \cdot \bar{S}_{n-1}$$

5.3 Aritmetica în Complement față de 2

Depășirea la adunare cu semn (cont)

	A	B	Condiție depășire	A_{n-1}	B_{n-1}	C_{n-1}	C_n	S_{n-1}	Eveniment
$A + B$	≥ 0	≥ 0	$S < 0$	0	0	0	0	0	
				0	0	1	0	1	depășire
	< 0	< 0	$S \geq 0$	1	1	0	1	0	depășire
				1	1	1	1	1	

Exemplele anterioare

Concluzie: $Ovr = C_{n-1} \oplus C_n$

• Fără depășire

Transportul

$$\begin{array}{rcl}
 & \xrightarrow{\quad} & \begin{array}{c} C_n C_{n-1} \\ \boxed{00}0011110 \end{array} \\
 13 & \Rightarrow & 00001101+ \\
 7 & \Rightarrow & 00000111 \\
 \hline
 20 & \leq & 00010100
 \end{array}$$

$$\begin{array}{rcl}
 & \xrightarrow{\quad} & \begin{array}{c} C_n C_{n-1} \\ \boxed{11}11110010 \end{array} \\
 13 & \Rightarrow & 00001101+ \\
 -7 & \Rightarrow & 11111001 \\
 \hline
 6 & \leq & 00000110
 \end{array}$$

• Cu depășire

Transportul

$$\begin{array}{rcl}
 & \xrightarrow{\quad} & \begin{array}{c} C_n C_{n-1} \\ \boxed{01}110 \end{array} \\
 3 & \Rightarrow & 0011+ \\
 7 & \Rightarrow & 0111 \\
 \hline
 -6 & \leq & 1010
 \end{array}$$

$$\begin{array}{rcl}
 & \xrightarrow{\quad} & \begin{array}{c} C_n C_{n-1} \\ \boxed{10}010 \end{array} \\
 -3 & \Rightarrow & 1101+ \\
 -7 & \Rightarrow & 1001 \\
 \hline
 6 & \leq & 0110
 \end{array}$$

5.4 Aritmetica în MIPS

Depășirea la adunare fără semn

Exemple

Cu depășire

Transportul

$$\begin{array}{rcl} & \xrightarrow{\quad} & C_n \\ & & \boxed{1}1100 \\ 3 & \Rightarrow & 0011+ \\ \underline{14} & \Rightarrow & \underline{1110} \\ 1 & \leq & 0001 \end{array}$$

Fără depășire

$$\begin{array}{rcl} & & C_n \\ & & \boxed{0}1110 \\ 3 & \Rightarrow & 0011+ \\ \underline{7} & \Rightarrow & \underline{0111} \\ 10 & \leq & 1010 \end{array}$$

Concluzie: $Ovr = C_n$

Abordarea MIPS

- La operații cu semn se detectează depășirea (**nu și la laborator!**).
 - ADD, SUB, ADDI
- La operații fără semn nu se detectează depășirea.
 - ADDU, SUBU, ADDIU



5.5 Proiectarea ALU

Cerințe

Instrucțiune	Operație ALU
ADD	adunare cu semn
SUB	scădere cu semn
AND	ȘI logic
OR	OR logic
ADDI	adunare cu semn
ORI	OR logic
LW	adunare cu semn
SW	adunare cu semn
BEQ	scădere cu semn
J	nu contează

Intrări: 2 operanzi pe 32 de biți.

Ieșiri: rezultat pe 32 biți, transport, semnale de stare (flags).

Operații: adunare, scădere, ȘI, SAU, cu posibilități de extindere.

Proiectare ALU MIPS = Proiectare mixtă

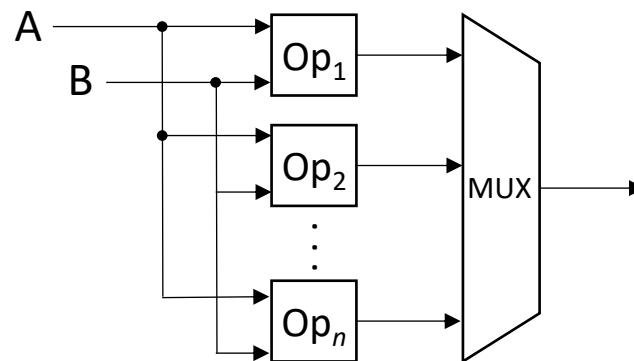
- folosește 2 abordări complementare.

Abordarea **Divide et Impera**

- Se implementează ALU pe 1 bit.
- Se extinde apoi la 32 biți prin cascada.

Abordarea **incrementală**

- Operațiile pe 1 bit sunt implementate pe rând, începând de la porți logice fundamentale: AND, OR, NOT, XOR.
- Gruparea operațiilor se face cu ajutorul unui MUX.



5.5 Proiectarea ALU

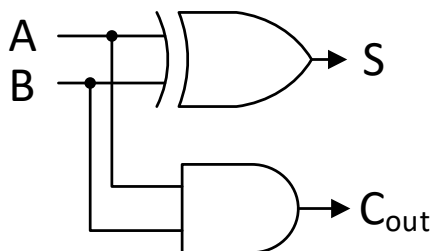
ALU pe 1 bit

- **Operațiile logice** - se pot implementa direct cu porți logice fundamentale.
- **Adunarea pe 1 bit**

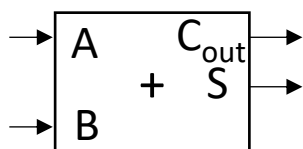
Semi-sumator pe 1 bit

A	B	C _{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$S = A \oplus B, C_{out} = A \cdot B$$



Simbol

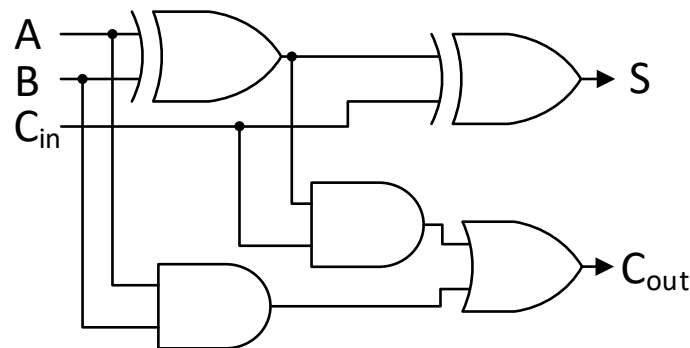


Sumator complet pe 1 bit

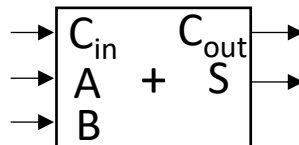
A	B	C _{in}	C _{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$S = A \oplus B \oplus C - \text{nr. impar de 1}$$

$$\begin{aligned} C_{out} &= A \cdot B + A \cdot C_{in} + B \cdot C_{in} = \\ &= A \cdot B + (A+B) \cdot C_{in} = \\ &= A \cdot B + (A \cdot B + A \oplus B) \cdot C_{in} = \\ &= A \cdot B + A \cdot B \cdot C_{in} + (A \oplus B) \cdot C_{in} = \\ &= A \cdot B \cdot (1 + C_{in}) + (A \oplus B) \cdot C_{in} = \\ &= A \cdot B + (A \oplus B) \cdot C_{in} \end{aligned}$$



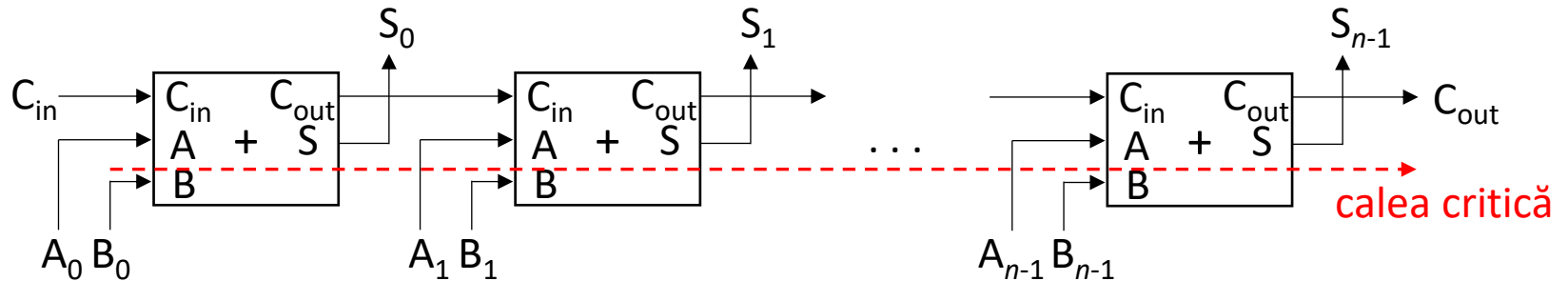
Simbol



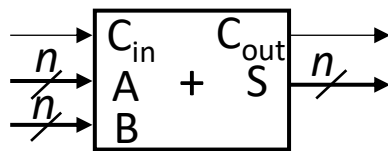
5.5 Proiectarea ALU

Sumatorul pe n biți

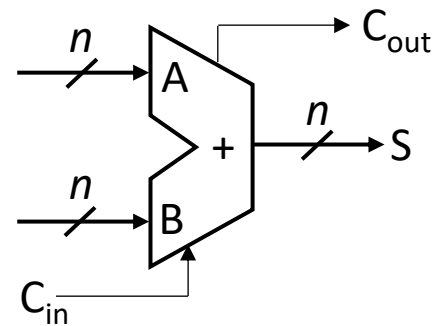
Se realizează prin cascada



Simbol



sau



5.5 Proiectarea ALU

ALU pe 1 bit

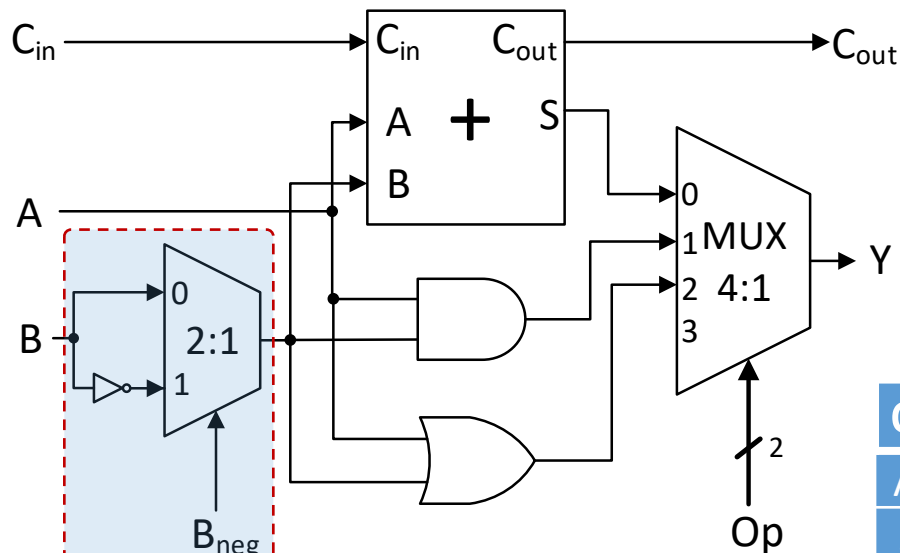
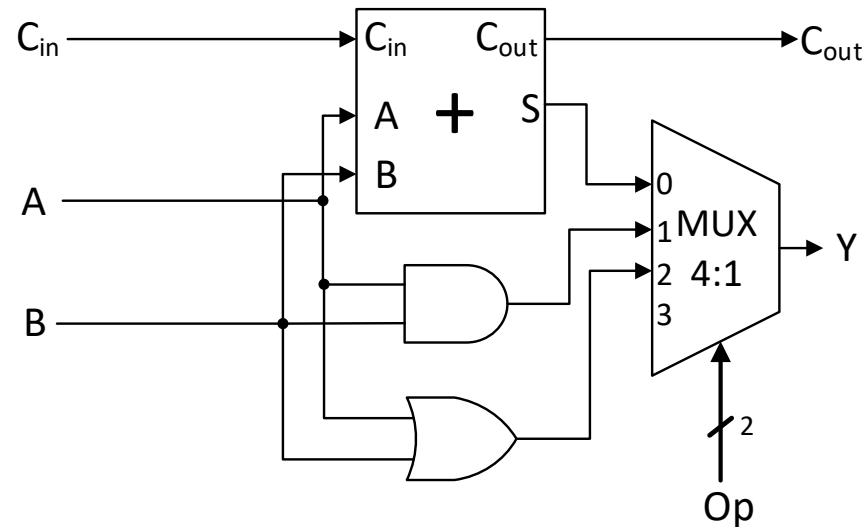
• Compunerea operațiilor:

▪ Sumator + ȘI + SAU

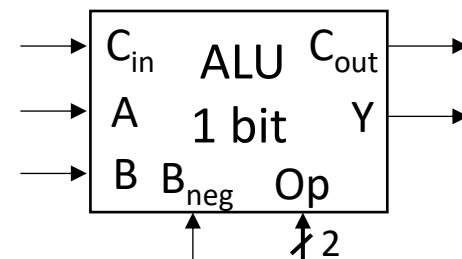
Notă: Se folosește MUX pentru selecția rezultatului.

▪ Sumator + Scăzător + ȘI + SAU

Obs: $A - B = A + (-B) = A + \overline{\overline{B}} = A + \overline{B} + 1$



Simbol



Operație	B_{neg}	$Op_{1:0}$		C_{in0}
Adunare	0	0	0	0
Scădere	1	0	0	1
ȘI	0	0	1	x
SAU	0	1	0	x

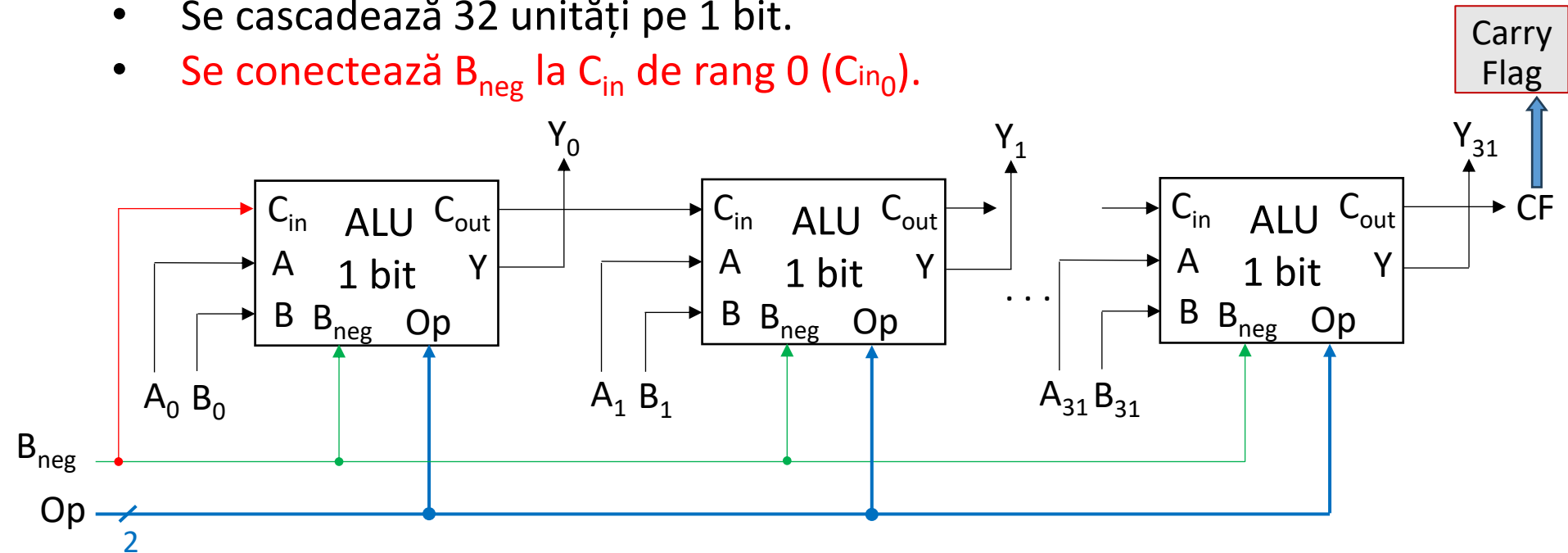
Obs:

$B_{neg} = C_{in0}$

5.5 Proiectarea ALU

ALU pe 32 de biți

- Se cascadează 32 unități pe 1 bit.
- Se conectează B_{neg} la C_{in} de rang 0 (C_{in0}).



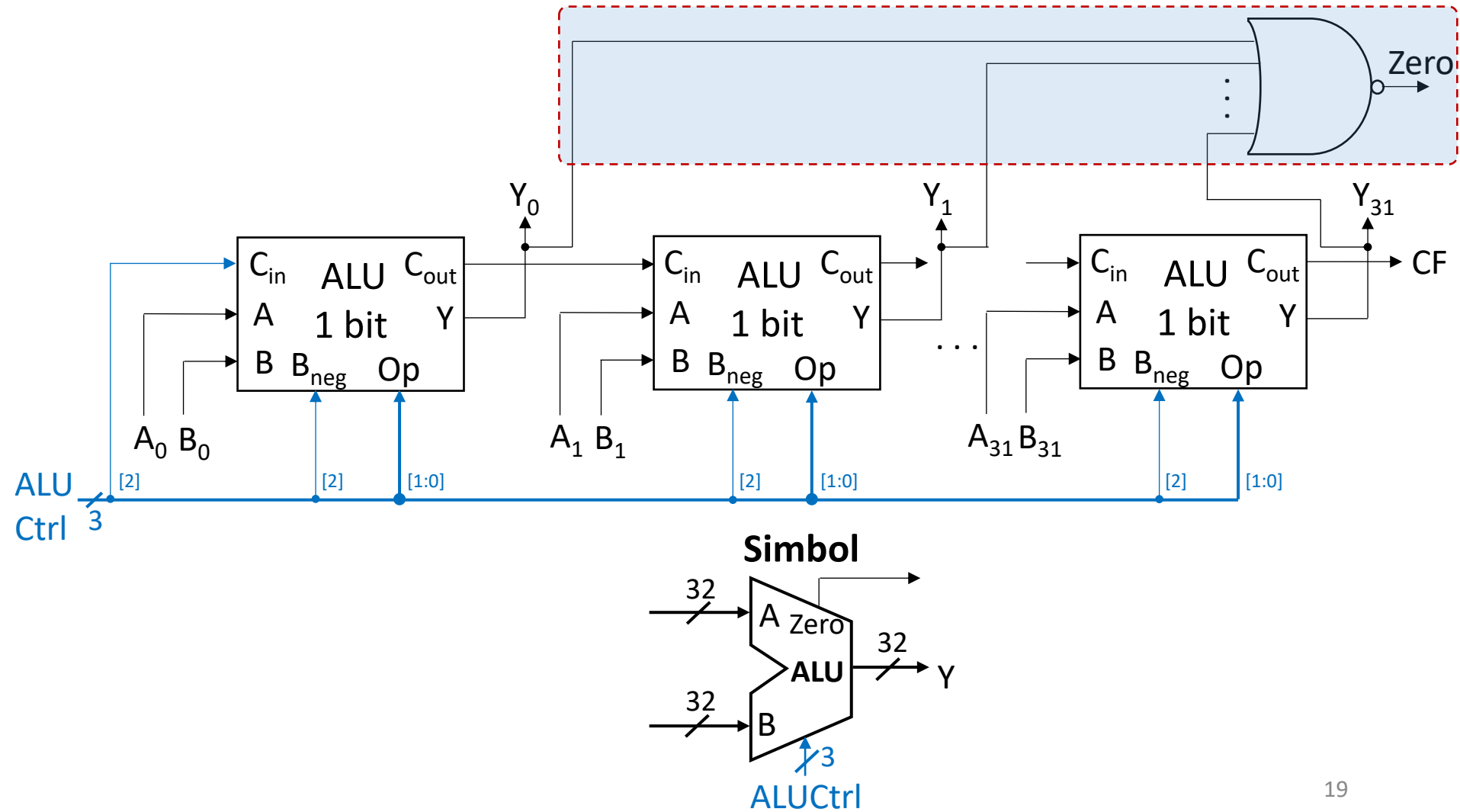
Operație	ALU Ctrl _{2:0}		
	B _{neg}	Op _{1:0}	
Adunare	0	0	0
Scădere	1	0	0
ȘI	0	0	1
SAU	0	1	0

$\Rightarrow \text{ALU Ctrl}_{2:0} \leftarrow B_{neg} || \text{Op}_{1:0}$

5.5 Proiectarea ALU

ALU pe 32 de biți, extindere (cont)

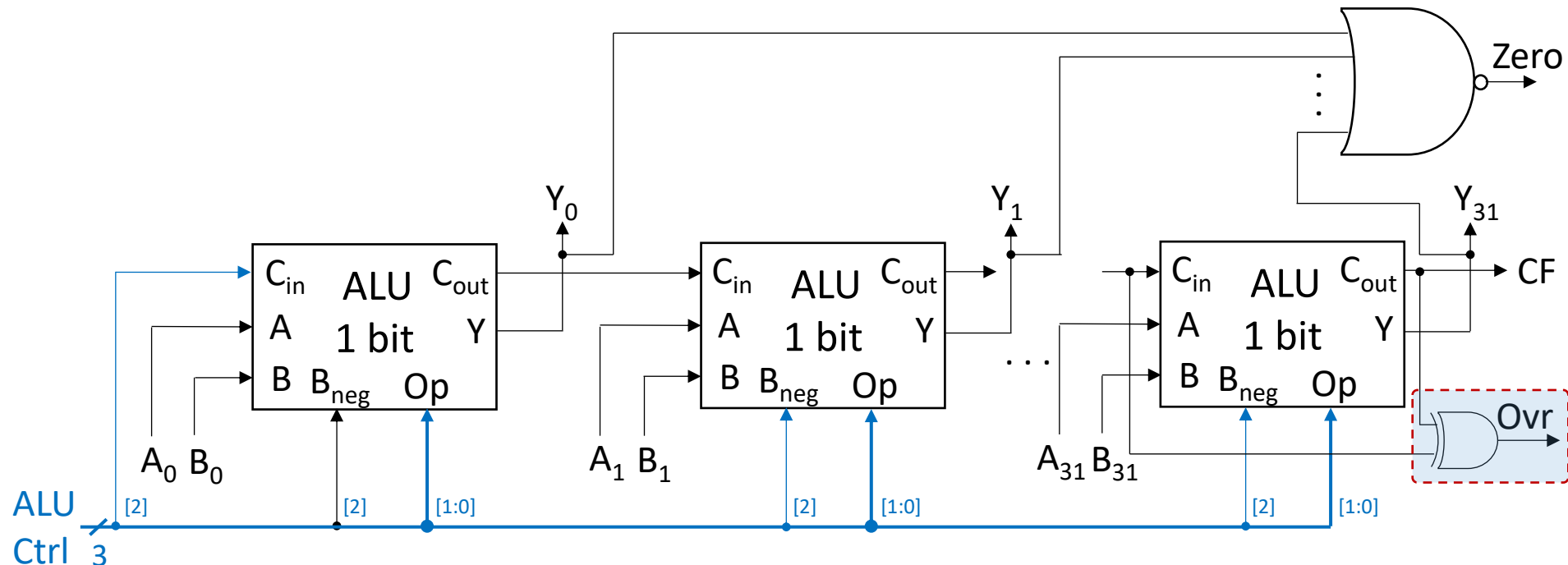
- Detecția unui rezultat nul se face cu NOR: $\text{Zero} \leftarrow \text{NOR}(Y_0, Y_1, \dots, Y_{31})$.



5.5 Proiectarea ALU

ALU pe 32 de biți, extindere (cont)

- Detecția depășirii (se ignoră la operații fără semn): $Ovr \leftarrow Cin_{31} \oplus Cout_{31}$.

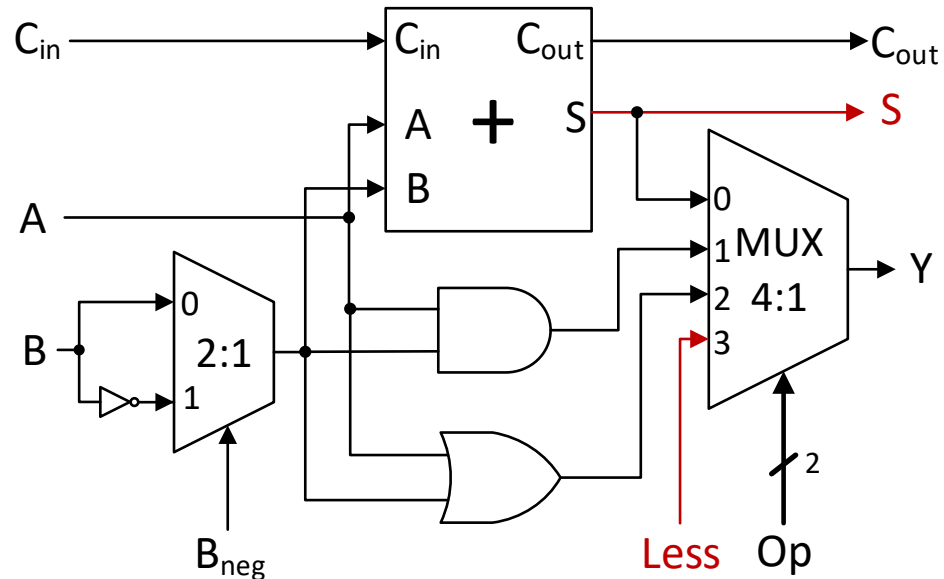


5.5 Proiectarea ALU

ALU pe 32 de biți, extindere (cont)

- Implementarea SLT: if $A < B$ then $Y \leftarrow 00...01_2$
else $Y \leftarrow 00...00_2$

Modificări
necesare la
ALU pe 1 bit
(cu roșu)



Obs: Adăugăm intrarea **Less**, asociată cu operația SLT și amplasăm **S** la ieșire.

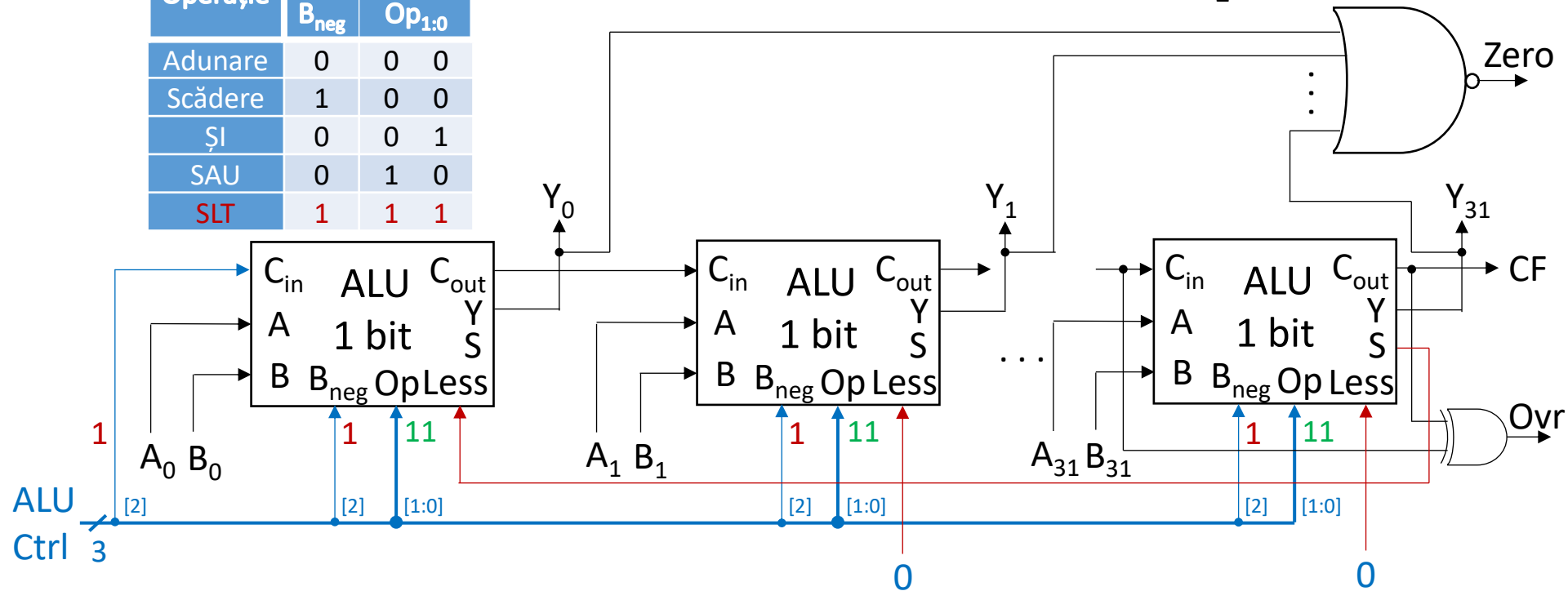
- Implementarea SLT în ALU pe 1 bit presupune $Y \leftarrow Less \Rightarrow Op = 11$

5.5 Proiectarea ALU

ALU pe 32 de biți, extindere (cont)

- Implementarea SLT (continuare): if $A < B$ then $Y \leftarrow 00...01_2$
else $Y \leftarrow 00...00_2$

Operație	ALUCtrl _{2:0}		
	B _{neg}	Op _{1:0}	
Adunare	0	0 0	
Scădere	1	0 0	
ȘI	0	0 1	
SAU	0	1 0	
SLT	1	1 1	



- Deoarece $Y \leftarrow 00...01_2$ sau $Y \leftarrow 00...00_2 \Rightarrow Less_i \leftarrow 0$, pentru $i = 1, 31$
- Setăm $B_{neg} = 1 \Rightarrow C_{in}_0 = 1 \Rightarrow S \leftarrow A + \bar{B} + 1 = A - B$ (se realizează scădere pe S)
- Obs:** if $A < B$ then $S < 0 \Rightarrow S_{31} = 1$
else $S \geq 0 \Rightarrow S_{31} = 0 \Rightarrow Less_0 \leftarrow S_{31}$
- Cod final **ALUCtrl_{2:0} = 111**

5.5 Proiectarea ALU

ALU pe 32 de biți, extindere (cont)

- Înmulțirea pe 32 de biți (MULT, MULTU – fără semn):
Este o adunare repetată a deînmulțitului (T1) deplasat progresiv la stânga, în funcție de valoarea biților din înmulțitor (T2).
 - Înmulțitor_i = 0 -> se adună 0000...0;
 - Înmulțitor_i = 1 -> se adună deînmulțitul deplasat.

Obs: Înmulțirea = deplasări și adunări repetate.

Ex: 1000x
 1001
 1000+
 0000
 0000
 1000
 01001000

Metode de implementare:

- Combinational;
- Pipeline;
- Multi-ciclu (iterativ, pași de adunare + deplasare).

Pași la înmulțirea cu semn MULT (la MULTU doar Pas₁):

- Înmulțirea fără semn pe valorile absolute: $res \leftarrow |T1| \times |T2|$;
- Se determină semnul rezultatului din semnele termenilor:
 $semn_{res} \leftarrow T1_{31} \oplus T2_{31}$;
- ($semn_{res}=1$) $\rightarrow res \leftarrow 0 - res = \overline{res} + 1$.

5.5 Proiectarea ALU

ALU pe 32 de biți, extindere (cont)

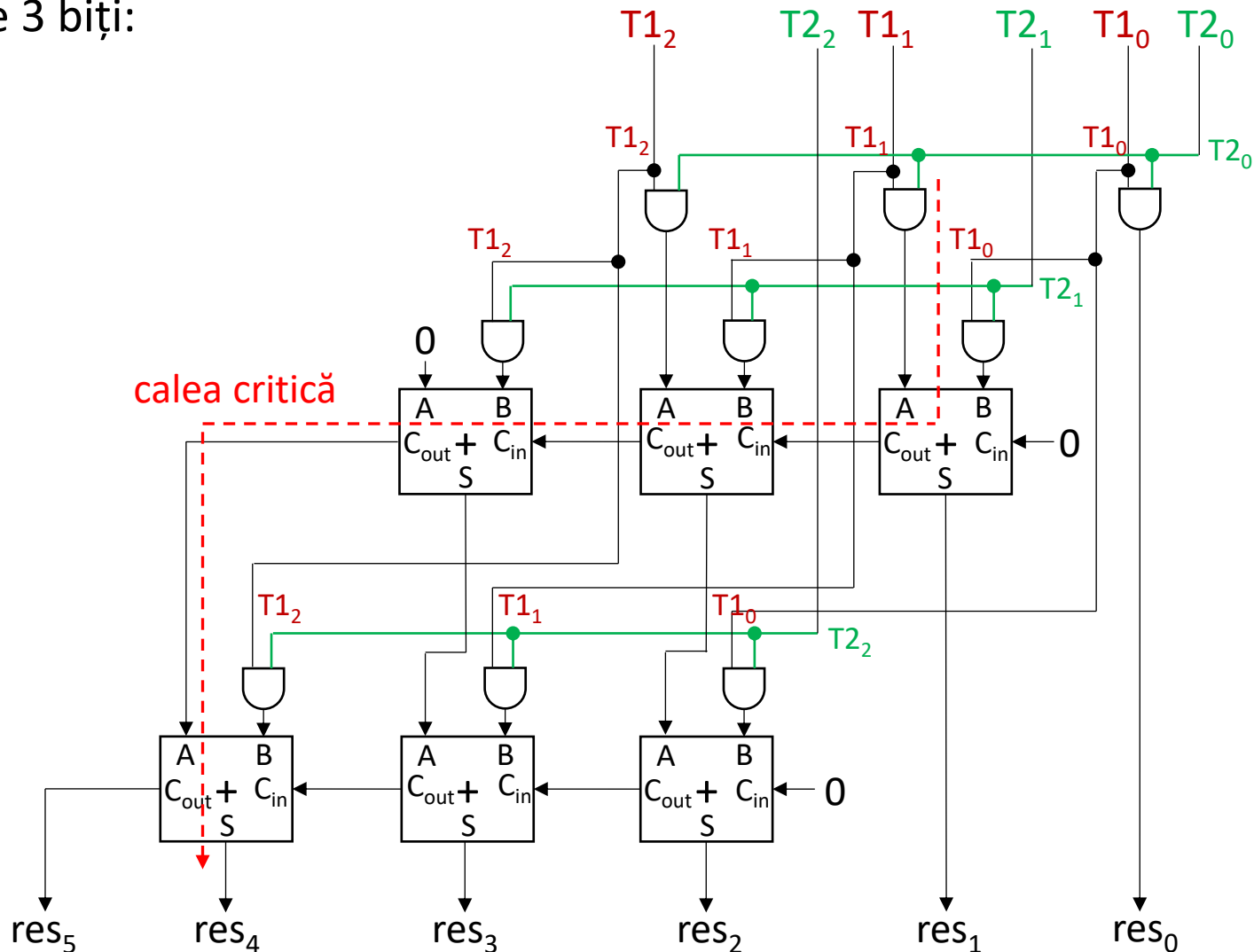
- Înmulțirea fără semn (la Pas_1), **combinational**:

Ex. pe 3 biți:

Se consideră:

$$T1_{2:0} = |T1|$$

$$T2_{2:0} = |T2|$$



5.5 Proiectarea ALU

ALU pe 32 de biți, extindere (cont)

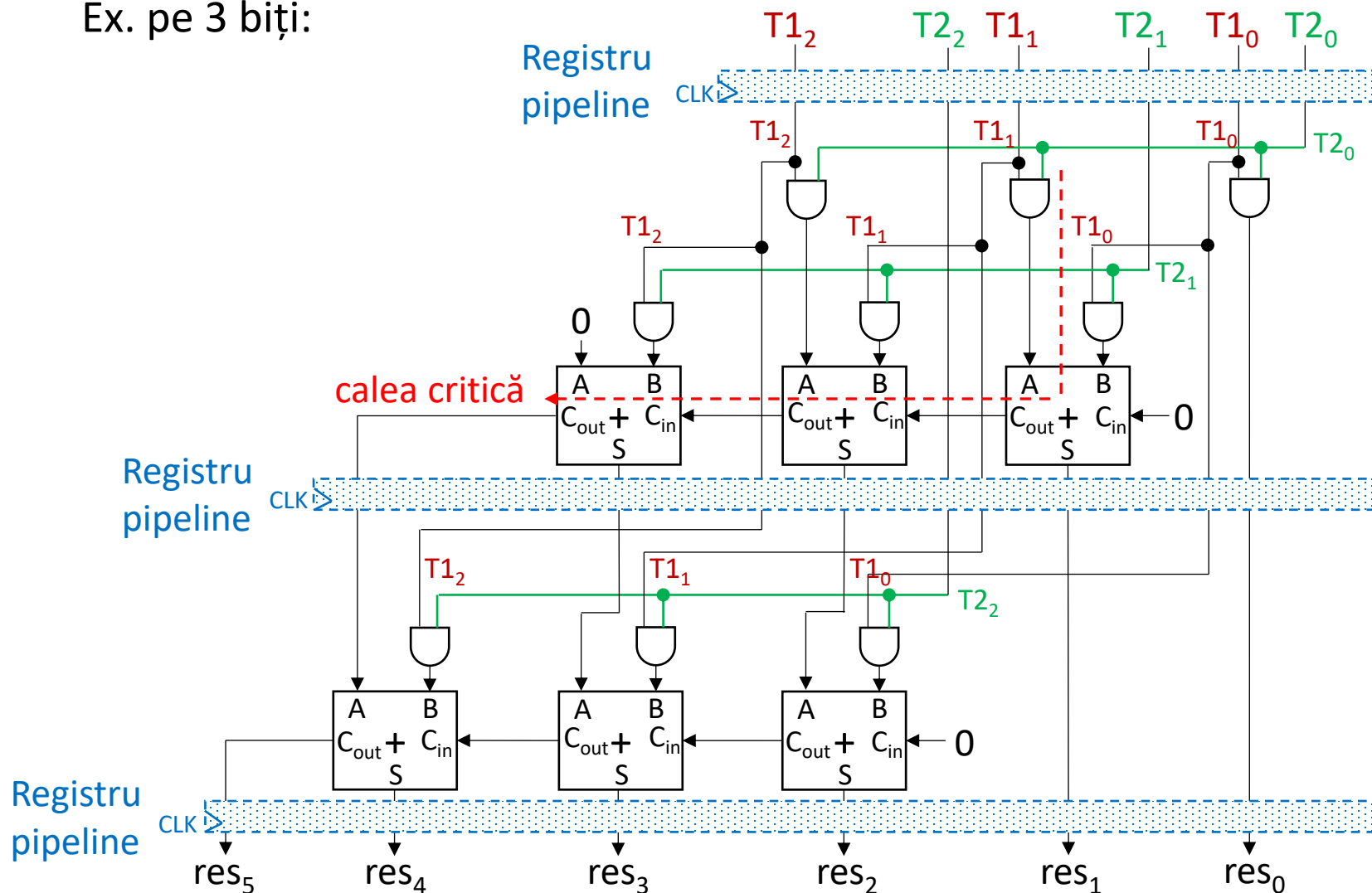
- Înmulțirea fără semn (la Pas_1), **pipeline**:

Ex. pe 3 biți:

Se consideră:

$$T1_{2:0} = |T1|$$

$$T2_{2:0} = |T2|$$



5.5 Proiectarea ALU

Se consideră:
 $T1 = |T1|$
 $T2 = |T2|$

ALU pe 32 de biți, extindere (cont)

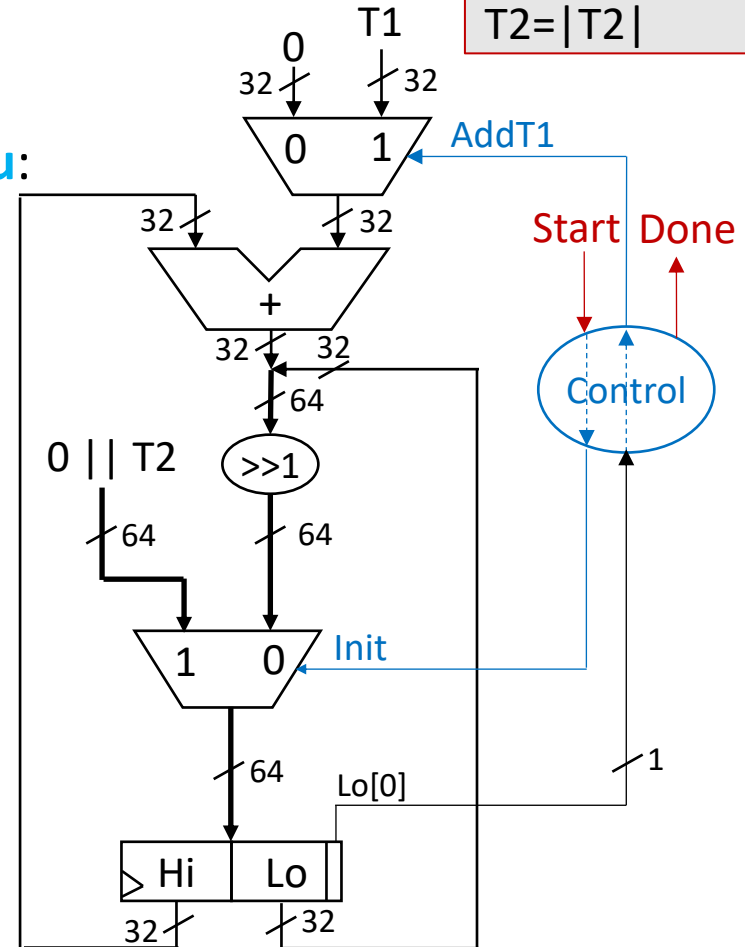
- Înmulțirea fără semn (la Pas_1), **multi-ciclu**:

Algoritm:

Start: $Hi_Lo \leftarrow 0 || T2$
 for 1 to 32 loop
 if $Lo[0]=1$ then $Hi_Lo \leftarrow ((Hi+T1) || Lo) \gg 1$
 else $Hi_Lo \leftarrow ((Hi + 0) || Lo) \gg 1$
Done $\leftarrow 1$; -- *rezultatul este în Hi Lo pe 64 biți*

Funcțiile unității **Control**:

- Init** $\leftarrow$ **Start**
- AddT1** $\leftarrow Lo[0]$
- Done** $\leftarrow 1$ după 32 iterații: se poate folosi un counter, inițializat sincron cu valoarea 32 de comanda **Start**, care se decrementează la fiecare ciclu $\Rightarrow$ **Done** $\leftarrow$ Zero_Det(counter)



5.5 Proiectarea ALU

ALU pe 32 de biți, extindere (cont)

- Înmulțirea fără semn (la Pas_1), **multi-ciclu**:

Algorithm:

Start: $Hi_Lo \leftarrow 0 \ || \ T2$
 for 1 to 32 loop
 if $Lo[0]=1$ then $Hi_Lo \leftarrow ((Hi+T1) \ || \ Lo) \gg 1$
 else $Hi_Lo \leftarrow ((Hi + 0) \ || \ Lo) \gg 1$
Done $\leftarrow 1$ -- *rezultatul este în Hi Lo pe 64 biți*

Ex. pe 4 biți (pt. simplificare) => se fac 4 iterații, în loc de 32:

$T1 = 0100$ x

$T2 = 0101$

Tabelul de trasare a execuției:

Pas	Hi Lo	Control	Efect (valoarea pregătită pentru scriere)
Start	XXXX XXXX	Init	$Hi_Lo \leftarrow 0000 \ 0101$
Iter1	0000 0101	addT1	$Hi_Lo \leftarrow ((0000+0100) \ \ 0101) \gg 1 = 0 \ 0100 \ 010$
Iter2	0010 0010		$Hi_Lo \leftarrow ((0010+0000) \ \ 0010) \gg 1 = 0 \ 0010 \ 001$
Iter3	0001 0001	addT1	$Hi_Lo \leftarrow ((0001+0100) \ \ 0001) \gg 1 = 0 \ 0101 \ 000$
Iter4	0010 1000		$Hi_Lo \leftarrow ((0010+0000) \ \ 1000) \gg 1 = 0 \ 0010 \ 100$
	0001 0100	Done	

5.5 Proiectarea ALU

ALU pe 32 de biți, extindere (cont)

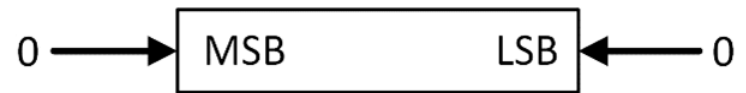
- Circuite de deplasare (SLL, SRA, SRL):

- Necesită posibilitatea de deplasare cu până la 31 biți în funcție de *shift amount*.

- Tipuri de deplasare:

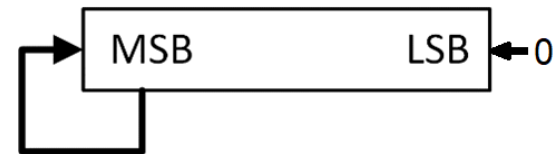
1. **Logică:**

- La stânga: cu 0
- La dreapta: cu 0



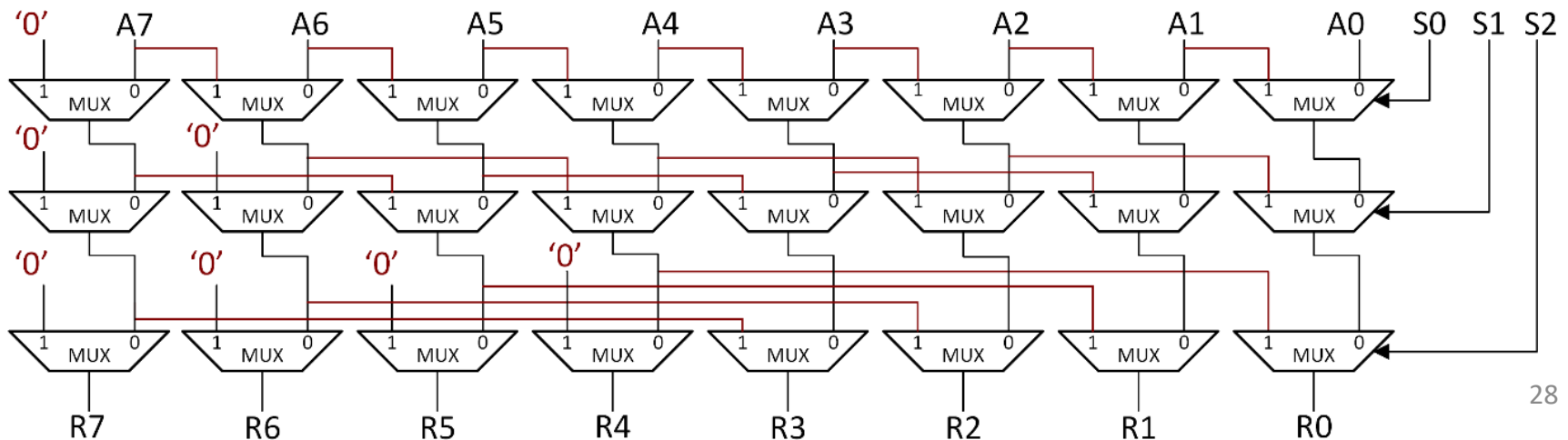
2. **Aritmetică:**

- La stânga: cu 0
- La dreapta: cu bitul de semn



- Tehnica de implementare **combi-națională** cu parametru de deplasare folosește MUX-uri cascade:

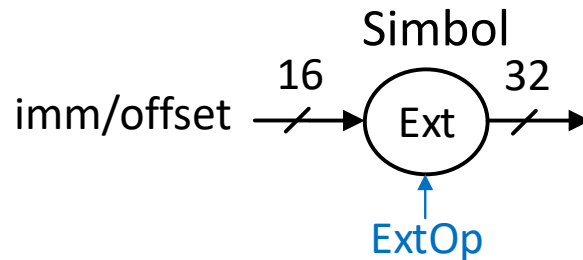
Ex. Deplasare la dreapta pe 8 biți, cu 3 biți de *shift amount* (deplasare cu max 7 poziții, $S_{2:0}=111$):



5.5 Proiectarea ALU

Cerințe adiționale

- Extindere operanzi (imediat / offset la instrucțiuni de tip I):
 - Extindere cu 0 (*logică*) – la operații logice: ANDI, ORI, XORI
 $RF[rt] \leftarrow RF[rs] \text{ op } Z_Ext(imm) - \text{operații logice}$
 - Extindere cu semn (*aritmetică*) – la restul operațiilor: ADDI, SUBI, LW, BEQ
 $RF[rt] \leftarrow \$s \text{ op } S_Ext(imm) - \text{operații aritmetice}$
 $RF[rt] \leftarrow Mem[RF[rs] + S_Ext(offset)] - LW, \text{ operații cu memoria}$
 $PC \leftarrow (PC+4)_{31:28} + (S_Ext(offset) \ll 2) - \text{salturi condiționate}$



- Dacă $ExtOp = 0 \Rightarrow$ extindere cu 0 (Z_Ext):
 $Z_Ext(imm) \leftarrow 0_{31} \dots 0_{16} imm_{15} \dots imm_0$
- Dacă $ExtOp = 1 \Rightarrow$ extindere cu semn (S_Ext):
 - Dacă $imm_{15} = 0 \Rightarrow S_Ext(imm) \leftarrow 0_{31} \dots 0_{16} imm_{15} \dots imm_0$
 - Dacă $imm_{15} = 1 \Rightarrow S_Ext(imm) \leftarrow 1_{31} \dots 1_{16} imm_{15} \dots imm_0$

Probleme propuse

1. Proiectați un ALU pe 8 biți care generează la ieșire următoarele valori: $A+B$, $A-B$, $A+1$, $A-1$, A și $\bar{A}$. Folosiți un singur sumator. Prezentați schema cu semnalele de control și un tabel cu valorile semnalelor de control pentru fiecare operație.
2. Prezentați algoritmul de înmulțire cu adunări și deplasări repetate. Dați un exemplu numeric.

Referințe

- [1] F. Oniga, “De la bit la procesor. Introducere în arhitectura calculatoarelor”, Cap. 2.2, 5.4, 5.5, 11, Ed. U.T. Press, 2019, ISBN 978-606-737-366-0.
- [2] D. A. Patterson, J. L. Hennessy, “Computer Organization and Design: The Hardware/Software Interface”, 5th edition, Cap. 3.2, 3.3, Anexa B5, Ed. Morgan–Kaufmann, 2013.
- [3] MIPS Technologies, Inc., “MIPS32™ Architecture for Programmers, Volume I: Introduction to the MIPS32™ Architecture”.
- [4] MIPS Technologies, Inc., “MIPS32™ Architecture for Programmers Volume II: The MIPS32™ Instruction Set”.